R Notebook

Code

3.2.5 Exercises

Q1)

Hide

```
library(dplyr)

# 1. Had an arrival delay of two or more hours
flights %>%
  filter(arr_delay >= 120)
```

y… <int>	mo… <int>	d.. <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>	arr_time <int>	sched_arr_time <int>	arr_delay <dbl>	c <chr>
2013	1	1	811	630	101	1047	830	137	M
2013	1	1	848	1835	853	1001	1950	851	M
2013	1	1	957	733	144	1056	853	123	U
2013	1	1	1114	900	134	1447	1222	145	U
2013	1	1	1505	1310	115	1638	1431	127	E
2013	1	1	1525	1340	105	1831	1626	125	B
2013	1	1	1549	1445	64	1912	1656	136	E
2013	1	1	1558	1359	119	1718	1515	123	E
2013	1	1	1732	1630	62	2028	1825	123	E
2013	1	1	1803	1620	103	2008	1750	138	M

1-10 of 10,200 rows | 1-10 of 19 columns Previous **1** 2 3 4 5 6 … 100 Next

Hide

```
# 2. Flew to Houston (IAH or HOU)
flights %>%
  filter(dest %in% c("IAH", "HOU"))
```

y… <int>	mo… <int>	d.. <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>	arr_time <int>	sched_arr_time <int>	arr_delay <dbl>	c <chr>
2013	1	1	517	515	2	830	819	11	U
2013	1	1	533	529	4	850	830	20	U
2013	1	1	623	627	-4	933	932	1	U

y… <int>	mo… <int>	d.. <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>	arr_time <int>	sched_arr_time <int>	arr_delay <dbl>	c <chr>
2013	1	1	728	732	-4	1041	1038	3	U
2013	1	1	739	739	0	1104	1038	26	U
2013	1	1	908	908	0	1228	1219	9	U
2013	1	1	1028	1026	2	1350	1339	11	U
2013	1	1	1044	1045	-1	1352	1351	1	U
2013	1	1	1114	900	134	1447	1222	145	U
2013	1	1	1205	1200	5	1503	1505	-2	U

Hide

```
# 3. Were operated by United, American, or Delta
flights %>%
  filter(carrier %in% c("UA", "AA", "DL"))
```

y… <int>	mo… <int>	d.. <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>	arr_time <int>	sched_arr_time <int>	arr_delay <dbl>	c <chr>
2013	1	1	517	515	2	830	819	11	U
2013	1	1	533	529	4	850	830	20	U
2013	1	1	542	540	2	923	850	33	A
2013	1	1	554	600	-6	812	837	-25	D
2013	1	1	554	558	-4	740	728	12	U
2013	1	1	558	600	-2	753	745	8	A
2013	1	1	558	600	-2	924	917	7	U
2013	1	1	558	600	-2	923	937	-14	U
2013	1	1	559	600	-1	941	910	31	A
2013	1	1	559	600	-1	854	902	-8	U

1-10 of 139,504 rows | 1-10 of 19 columns Previous **1** 2 3 4 5 6 … 100 Next

Hide

```
# 4. Departed in summer (July, August, and September)
flights %>%
  filter(month %in% c(7, 8, 9))
```

y…	mo…	d..	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	c
<int>	<int>	<int>	<int>	<int>	<dbl>	<int>	<int>	<dbl>	<
2013	7	1	1	2029	212	236	2359	157	B
2013	7	1	2	2359	3	344	344	0	B
2013	7	1	29	2245	104	151	1	110	B
2013	7	1	43	2130	193	322	14	188	B
2013	7	1	44	2150	174	300	100	120	A
2013	7	1	46	2051	235	304	2358	186	B
2013	7	1	48	2001	287	308	2305	243	V
2013	7	1	58	2155	183	335	43	172	B
2013	7	1	100	2146	194	327	30	177	B
2013	7	1	100	2245	135	337	135	122	B

1-10 of 86,326 rows | 1-10 of 19 columns Previous **1** 2 3 4 5 6 … 100 Next

Hide

```
# 5. Arrived more than two hours late but didn’t leave late
flights %>%
  filter(arr_delay > 120, dep_delay <= 0)
```

y…	mo…	d..	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	c
<int>	<int>	<int>	<int>	<int>	<dbl>	<int>	<int>	<dbl>	<
2013	1	27	1419	1420	-1	1754	1550	124	M
2013	10	7	1350	1350	0	1736	1526	130	E
2013	10	7	1357	1359	-2	1858	1654	124	A
2013	10	16	657	700	-3	1258	1056	122	B
2013	11	1	658	700	-2	1329	1015	194	V
2013	3	18	1844	1847	-3	39	2219	140	U
2013	4	17	1635	1640	-5	2049	1845	124	M
2013	4	18	558	600	-2	1149	850	179	A
2013	4	18	655	700	-5	1213	950	143	A
2013	5	22	1827	1830	-3	2217	2010	127	M

1-10 of 29 rows | 1-10 of 19 columns Previous **1** 2 3 Next

Hide

```
# 6. Were delayed by at least an hour, but made up over 30 minutes in flight
flights %>%
  filter(dep_delay >= 60, dep_delay - arr_delay > 30)
```

y… <int>	mo… <int>	d.. <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>	arr_time <int>	sched_arr_time <int>	arr_delay <dbl>	c <chr>
2013	1	1	2205	1720	285	46	2040	246	A
2013	1	1	2326	2130	116	131	18	73	B
2013	1	3	1503	1221	162	1803	1555	128	U
2013	1	3	1839	1700	99	2056	1950	66	A
2013	1	3	1850	1745	65	2148	2120	28	A
2013	1	3	1941	1759	102	2246	2139	67	U
2013	1	3	1950	1845	65	2228	2227	1	B
2013	1	3	2015	1915	60	2135	2111	24	9
2013	1	3	2257	2000	177	45	2224	141	9
2013	1	4	1917	1700	137	2135	1950	105	A

1-10 of 1,844 rows | 1-10 of 19 columns Previous **1** 2 3 4 5 6 … 100 Next

Hide

```
NA
```

Q2)

Hide

```
flights %>%
  arrange(desc(dep_delay))
```

y… <int>	mo… <int>	d.. <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>	arr_time <int>	sched_arr_time <int>	arr_delay <dbl>	c <chr>
2013	1	9	641	900	1301	1242	1530	1272	H
2013	6	15	1432	1935	1137	1607	2120	1127	M
2013	1	10	1121	1635	1126	1239	1810	1109	M
2013	9	20	1139	1845	1014	1457	2210	1007	A
2013	7	22	845	1600	1005	1044	1815	989	M
2013	4	10	1100	1900	960	1342	2211	931	D
2013	3	17	2321	810	911	135	1020	915	D

y… <int>	mo… <int>	d.. <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>	arr_time <int>	sched_arr_time <int>	arr_delay <dbl>	c <chr>
2013	6	27	959	1900	899	1236	2226	850	D
2013	7	22	2257	759	898	121	1026	895	D
2013	12	5	756	1700	896	1058	2020	878	A

Hide

```
flights %>%
  arrange(dep_time)
```

y… <int>	mo… <int>	d.. <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>	arr_time <int>	sched_arr_time <int>	arr_delay <dbl>	c <chr>
2013	1	13	1	2249	72	108	2357	71	B
2013	1	31	1	2100	181	124	2225	179	W
2013	11	13	1	2359	2	442	440	2	B
2013	12	16	1	2359	2	447	437	10	B
2013	12	20	1	2359	2	430	440	-10	B
2013	12	26	1	2359	2	437	440	-3	B
2013	12	30	1	2359	2	441	437	4	B
2013	2	11	1	2100	181	111	2225	166	W
2013	2	24	1	2245	76	121	2354	87	B
2013	3	8	1	2355	6	431	440	-9	B

1-10 of 336,776 rows | 1-10 of 19 columns Previous **1** 2 3 4 5 6 … 100 Next

Hide

```
NA
```

Q4)

Hide

```
flights %>%
  count(year, month,day) %>%
  summarise(all_days = all(n>0))
```

all_days <lgl>
TRUE

1 row

Hide

```
NA
NA
NA
```

3.3.5 Exercise

Q1)

dep_delay = dep_time - shed_dep_time dep_delay is the difference between the scheduled departure time and departure time indicating the difference or the delay in the departure of the flight.

Q2)

Select command extracts the column which are specified from the entire dataset. So despite of repeating the same column name withing the select() statement it will extract that column only once and automatically ignore the repeatation. As demonstrated below

Hide

```
flights %>%
  select(dep_delay, dep_delay, dep_delay)%>%
  select(dep_delay)
```

dep_delay <dbl>
2
4
2
-1
-6
-4
-5
-3
-3
-2

1-10 of 336,776 rows Previous **1** 2 3 4 5 6 … 100 Next

Hide

```
NA
```

Q3)

any_of() lets me pass the vector column names to select(). This se;ect all those columns that are present in the dataset and ignore the ones which are not. Now in this : variables <- c(“year”, “month”, “day”, “dep_delay”, “arr_delay”)

Q4)

on using the any_of() within select will help us select all those which are present in the Flights table. As shown below all the columns that are present in flights table are shown in the output. Here in this case all the columns are present in the flights table.

Hide

```
variables <- c("year", "month", "day", "dep_delay", "arr_delay")

flights %>%
  select(any_of(variables))
```

year <int>	month <int>	day <int>	dep_delay <dbl>	arr_delay <dbl>
2013	1	1	2	11
2013	1	1	4	20
2013	1	1	2	33
2013	1	1	-1	-18
2013	1	1	-6	-25
2013	1	1	-4	12
2013	1	1	-5	19
2013	1	1	-3	-14
2013	1	1	-3	-8
2013	1	1	-2	8

1-10 of 336,776 rows Previous **1** 2 3 4 5 6 … 100 Next

Q5)

Yes on running this code I am surprised to see that any column name that consits of ‘time’ irrespective of it being a prefix, suffix, uppercase or lower case, the results have shown all the columns consisting of the word ‘time’ is selected.

Hide

```
flights |> select(contains("TIME"))
```

dep_time <int>	sched_dep_time <int>	arr_time <int>	sched_arr_time <int>	air_time <dbl>	time_hour <S3: POSIXct>
517	515	830	819	227	2013-01-01 05:00:00
533	529	850	830	227	2013-01-01 05:00:00
542	540	923	850	160	2013-01-01 05:00:00
544	545	1004	1022	183	2013-01-01 05:00:00
554	600	812	837	116	2013-01-01 06:00:00
554	558	740	728	150	2013-01-01 05:00:00
555	600	913	854	158	2013-01-01 06:00:00
557	600	709	723	53	2013-01-01 06:00:00
557	600	838	846	140	2013-01-01 06:00:00
558	600	753	745	138	2013-01-01 06:00:00

1-10 of 336,776 rows Previous **1** 2 3 4 5 6 … 100 Next

Helpers like contains() in this case is by default case-sensitive. It will match any word that matches with the value inserted in it. Like “TIME” here, eventhough it is in all uppercase helpers are still able to match column names like sched_dep_time , arr_time. Both of these columns have ‘time’ term. The select() is used for selecting those columns which have been mentioned within the brackets. So on combining both select and helpers we can attain the desired results without worrying about case sensitivity.

we can change that default by: flights |> select(contains(“TIME”, ignore.case = FALSE) so here TIME would only match the ones with uppercase column names.

Hide

```
flights |> select(contains("TIME", ignore.case = FALSE))
```

336776 rows

Hide

```
NA
NA
```

Q6)

Hide

```
flights %>%
  rename(air_time_min = air_time )%>%
  relocate(air_time_min)
```

air_time_min <dbl>	y… <int>	mo… <int>	d.. <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>	arr_time <int>	sched_arr_tim <int
227	2013	1	1	517	515	2	830	81

air_time_min <dbl>	y… <int>	mo… <int>	d.. <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>	arr_time <int>	sched_arr_tim <int
227	2013	1	1	533	529	4	850	83
160	2013	1	1	542	540	2	923	85
183	2013	1	1	544	545	-1	1004	102
116	2013	1	1	554	600	-6	812	83
150	2013	1	1	554	558	-4	740	72
158	2013	1	1	555	600	-5	913	85
53	2013	1	1	557	600	-3	709	72
140	2013	1	1	557	600	-3	838	84
138	2013	1	1	558	600	-2	753	74

Hide

```
NA
NA
NA
NA
```

3.5.7

Q1

Hide

```
flights %>%
  group_by(carrier)%>%
  summarise(
    avg = mean(arr_delay, na.rm = TRUE),
    num = n()

  ) %>%
  arrange(desc(avg))
```

carrier <chr>	avg <dbl>	num <int>
F9	21.9207048	685
FL	20.1159055	3260
EV	15.7964311	54173
YV	15.5569853	601
OO	11.9310345	32
MQ	10.7747334	26397

carrier <chr>	avg <dbl>	num <int>
WN	9.6491199	12275
B6	9.4579733	54635
9E	7.3796692	18460
UA	3.5580111	58665

1-10 of 16 rows Previous **1** 2 Next

Q2)

Hide

```
flights %>%
  group_by(dest,flight)%>%
  summarise(
    delay = mean(dep_delay, na.rm = TRUE),

  )%>%
  arrange(desc(delay))
```

dest <chr>	flight <int>	delay <dbl>
CLE	372	483.00000
DEN	488	379.00000
DTW	5017	372.00000
BNA	521	346.00000
MCO	468	334.00000
CLE	3261	321.00000
SLC	809	298.00000
IAH	1510	278.00000
DEN	1275	277.00000
GSO	5478	242.00000

1-10 of 11,467 rows Previous **1** 2 3 4 5 6 ... 100 Next

Hide

```
NA
NA
NA
NA
```

Q5)

Count() groups the data by one or more variables and counts the number of rows in each group. Count() by default lists the result in the dataset's order. On adding sort = TRUE it sorts the results in descending order of n (largest first)

Hide

```
flights %>% count(carrier, sort = TRUE)
```

carrier <chr>	n <int>
UA	58665
B6	54635
EV	54173
DL	48110
AA	32729
MQ	26397
US	20536
9E	18460
WN	12275
VX	5162

1-10 of 16 rows Previous **1** 2 Next

Hide

```
NA
NA
```

Q6)

a.

group_by() divides the dataset into groups meaningful to analysis. group_by() adds grouped features to the data frame which changes behaviour of the subsequent verbs applied to the data. So on running df |> group_by(y) we will see a table where the data will be arranged according to y column.

Hide

```
df <- tibble(
  x = 1:5,
  y = c("a", "b", "a", "a", "b"),
  z = c("K", "K", "L", "L", "K")
)

df |> group_by(y)
```

x <int>	y <chr>	z <chr>
1	a	K
2	b	K
3	a	L
4	a	L
5	b	K

5 rows

Hide

```
NA
NA
```

b. I think the output of df |> arrange(y) will result in the sorting of column ‘y’ in ascending order making other columns rows to be arranged according to column ‘y’.

group_by() does not sort or change the values in the tibble instead adds grouped features to the data frame which changes behaviour of the subsequent verbs applied to the data. Where as arrange() is responsible for changing order of rows in the table. It just sorts the rows by column values without worrying about adding grouping info.

Hide

```
df |>
  arrange(y)
```

x <int>	y <chr>	z <chr>
1	a	K
3	a	L
4	a	L
2	b	K
5	b	K

5 rows

c. We are treating rows with the same ‘y’ column values as a group. so here there are 2 distinct values in column ‘y’ a and b. y = a –> rows (x = 1,3,4) and y = b –> rows (x= 2,5). Now for each group the summarize(mean_x = mean(x)) is calculating mean of x.

What the pipeline does?

1. Groups the dataset by y.
2. Summarizes each group by computing the mean of column x.
3. Result: a collapsed tibble with one row per y group.

Hide

```
df |>
  group_by(y) |>
  summarize(mean_x = mean(x))
```

y <chr>	mean_x <dbl>
a	2.666667
b	3.500000

2 rows

d. Here it is grouped by both the columns y and z. We are considering distinct values of both the columns so: So groups are defined by each unique combination of (y, z):

Group (a, K): row 1 → x = 1

Group (a, L): rows 3 & 4 → x = 3, 4

Group (b, K): rows 2 & 5 → x = 2, 5

and

summarize(mean_x = mean(x))

(a, K): mean(1) = 1.0

(a, L): mean(3, 4) = 3.5

(b, K): mean(2, 5) = 3.5

What the pipeline does?

Groups the data by both y and z. Computes the mean of x for each group. Returns one row per unique (y, z) combination.

Hide

```
df |>
  group_by(y, z) |>
  summarize(mean_x = mean(x))
```

y <chr>	z <chr>	mean_x <dbl>
a	K	1.0
a	L	3.5
b	K	3.5

3 rows

e. Here : group_by(y, z)

Groups by the pair (y, z):

(a, K): row 1 → x = 1

(a, L): rows 3 & 4 → x = 3, 4

(b, K): rows 2 & 5 → x = 2, 5

summarize(mean_x = mean(x), .groups = “drop”)

(a, K): mean(1) = 1.0

(a, L): mean(3, 4) = 3.5

(b, K): mean(2, 5) = 3.5

.groups = “drop” tells dplyr to remove all grouping after summarization.

What the pipeline does?

Groups rows by the combination of y and z. Computes the mean of x within each group. Returns one row per unique (y, z) pair. Drops all grouping information afterwards.

Hide

```
df |>
  group_by(y, z) |>
  summarize(mean_x = mean(x), .groups = "drop")
```

y <chr>	z <chr>	mean_x <dbl>
a	K	1.0
a	L	3.5
b	K	3.5

3 rows

Hide

```
NA
```

f. Pipeline 1: summarize()

Hide

```
df |>
  group_by(y, z) |>
  summarize(mean_x = mean(x))
```

y <chr>	z <chr>	mean_x <dbl>
a	K	1.0
a	L	3.5
b	K	3.5

3 rows

Groups by (y, z) → 3 groups:

(a, K): row 1 → x = 1

(a, L): rows 3, 4 → x = 3, 4

(b, K): rows 2, 5 → x = 2, 5

Summarize each group by mean of x:

(a, K) → 1

(a, L) → 3.5

(b, K) → 3.5

–> Collapses each group into one row per (y,z) combination.

Pipeline 2: mutate()

Hide

```
df |>
  group_by(y, z) |>
  mutate(mean_x = mean(x))
```

x <int>	y <chr>	z <chr>	mean_x <dbl>
1	a	K	1.0
2	b	K	3.5
3	a	L	3.5
4	a	L	3.5
5	b	K	3.5

5 rows

Groups are the same (a,K), (a,L), (b,K).

mutate() adds a new column mean_x, repeating the group mean across all rows in the group:

(a, K): row 1 → mean_x = 1

(a, L): rows 3, 4 → mean_x = 3.5

(b, K): rows 2, 5 → mean_x = 3.5

–> This keeps all original rows, but appends the group mean to each row.

difference between outputs:

summarize() –> collapses one row per group. and mutate() –> expands by keeping all rows, but appends a new column to the dataset.